CEG5101 Modern Computer Networking

Graduate Assistants: Ms. Poorvi Joshi (e1144005@u.nus.edu), Ms. Le Minh Chau (le_minhchau@u.nus.edu), Ms. Theepana Govintharajah (theepana@u.nus.edu)
Prepared by: Mr. Binghui Wu and Ms. Divya D Kulkarni (Ex - Graduate Assistants)
Lecturer: Assoc. Prof. Mohan Gurusamy
ECE Department, CDE., National University of Singapore

Laboratory 01
Introduction to Mininet

LEARNING OBJECTIVES

Upon successful completion of this laboratory, the students will be able to learn:

i. To download and install Mininet
ii. To become familiar with the Mininet Interface

EQUIPMENT

i. *Hardware* – Laptop/Computer with internet access
ii. *Software* – Mininet, Linux OS

DISCUSSION

Task 1: To download and install Mininet

1. Mininet is a network emulator for prototyping a large network on a single machine.

2. It is a virtual environment that allows network administrators to experiment and run a collection of network devices, such as routers, switches, and other related network devices on a Linux kernel.

3. Mininet is used to design simple as well as complex custom-defined topologies.

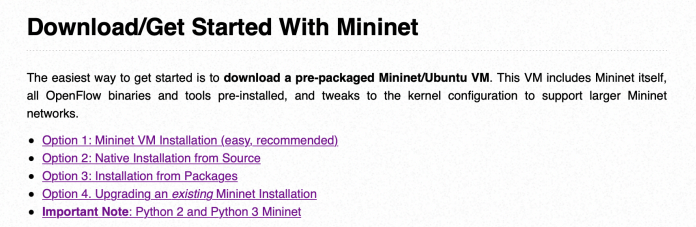
4. Mininet offers a user-friendly environment for development and research and allows users to deploy custom networks through the CLI and/or GUI interfaces.

5. Mininet is an open-source project, and its source code can be explored on https://github.com/mininet

6. To download and install Mininet, you need a Linux environment. NOTE: If you are not using Linux, follow and complete lab-00 first before proceeding to the next steps.

7. To download and get started with Mininet, proceed to http://mininet.org/download/ and choose "Option 2: Native Installation from Source" as shown below.

https://github.com/mininet/mininet/wiki/Introduction-to-Mininet
http://mininet.org/walkthrough/



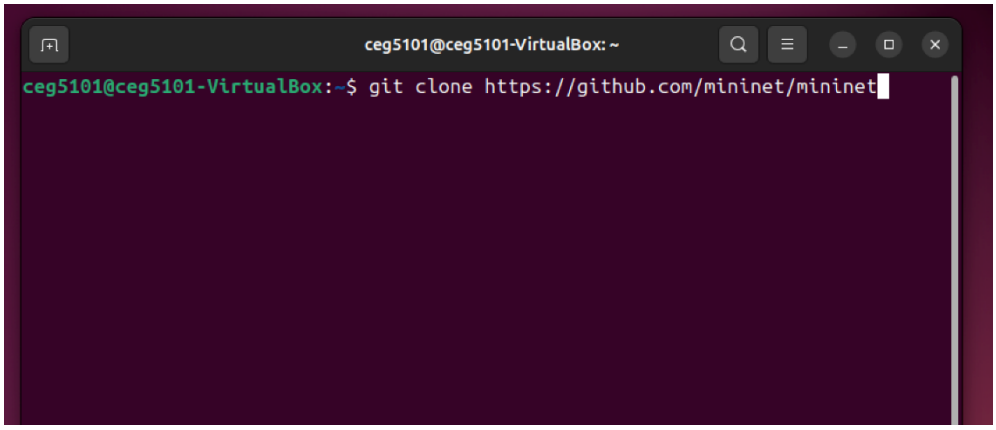
Download/Get Started With Mininet

The easiest way to get started is to **download a pre-packaged Mininet/Ubuntu VM**. This VM includes Mininet itself, all OpenFlow binaries and tools pre-installed, and tweaks to the kernel configuration to support larger Mininet networks.

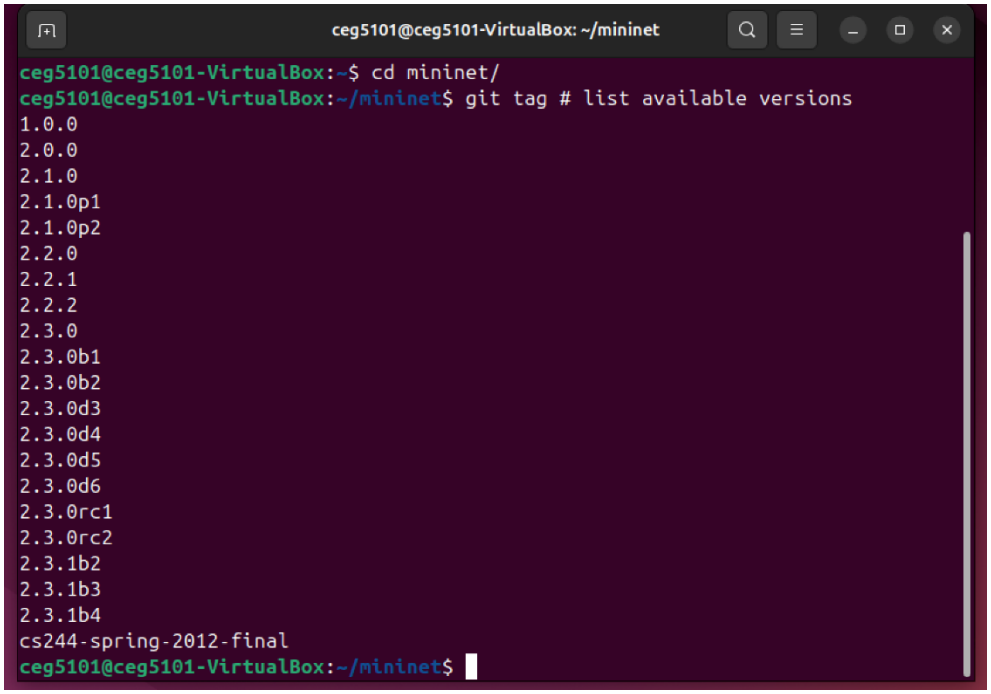
- Option 1: Mininet VM Installation (easy, recommended)
- Option 2: Native Installation from Source
- Option 3: Installation from Packages
- Option 4. Upgrading an *existing* Mininet Installation
- **Important Note**: Python 2 and Python 3 Mininet

8. Next, open the terminal on Ubuntu and follow the given steps to complete the installation process.

9. The first step is to get the source code using the command, " `git`
10. `https://github.com/mininet/mininet` "



11. Next, open the mininet folder (using the command "`cd mininet`") and check for the latest available version using the command "git tag # list available versions". You can continue with any version but it's recommended to install the latest one.



https://github.com/mininet/mininet/wiki/Introduction-to-Mininet
http://mininet.org/walkthrough/

12. Choose your version for installation with the command "`git checkout -b mininet-2.3.1b4 2.3.1b4`". Note: I have chosen the latest available version here.

```
2.3.0d4
2.3.0d5
2.3.0d6
2.3.0rc1
2.3.0rc2
2.3.1b2
2.3.1b3
2.3.1b4
cs244-spring-2012-final
ceg5101@ceg5101-VirtualBox:~/mininet$ git checkout -b mininet-2.3.1b4 2.3.1b4
Switched to a new branch 'mininet-2.3.1b4'
ceg5101@ceg5101-VirtualBox:~/mininet$
```

13. Next, go back to the source tree or home location of your PC with the command "`cd ..`".

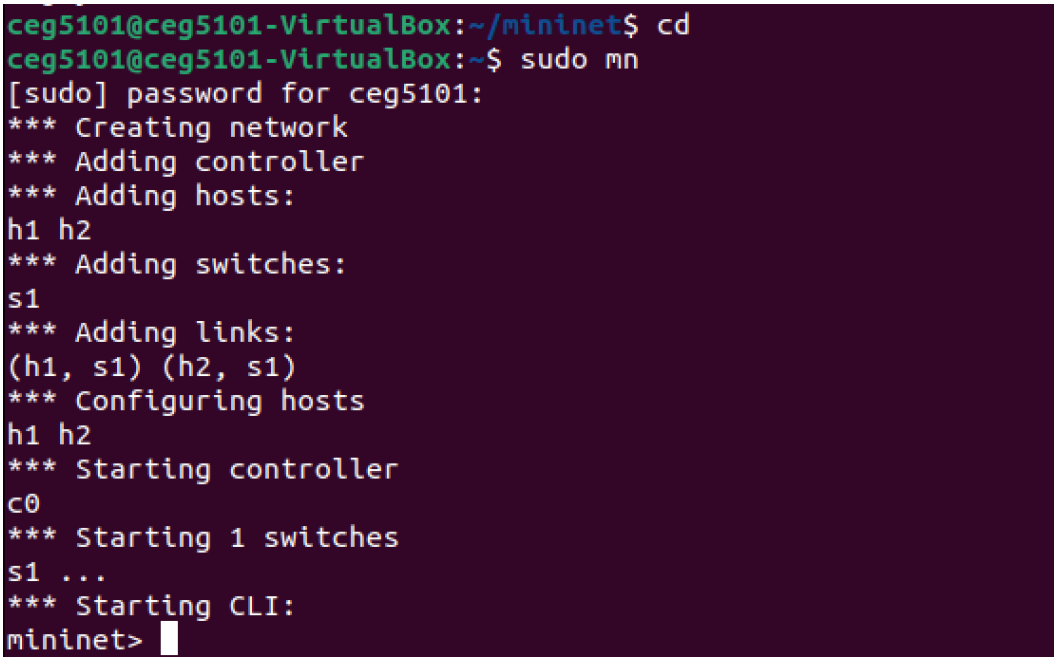
14. Once you are at the root location, then type the command "`~/mininet/util/install.sh`" to begin the installation process.

```
cs244-spring-2012-final
ceg5101@ceg5101-VirtualBox:~/mininet$ git checkout -b mininet-2.3.1b4 2.3.1b4
Switched to a new branch 'mininet-2.3.1b4'
ceg5101@ceg5101-VirtualBox:~/mininet$ cd ..
ceg5101@ceg5101-VirtualBox:~$ ~/mininet/util/install.sh
Detected Linux distribution: Ubuntu 24.04 noble amd64
sys.version_info(major=3, minor=12, micro=3, releaselevel='final', serial=0)
Detected Python (python3) version 3
Installing all packages except for -eix (doxypy, ivs, nox-classic)...
Install Mininet-compatible kernel if necessary
[sudo] password for ceg5101:
Hit:1 http://security.ubuntu.com/ubuntu noble-security InRelease
```

15. Once the installation is complete, you will see the process ended with a message "`Enjoy Mininet!`".

```
libtool: install: /usr/bin/install -c cbench /usr/local/bin/cbench
make[2]: Nothing to be done for 'install-data-am'.
make[2]: Leaving directory '/home/ceg5101/oflops/cbench'
make[1]: Leaving directory '/home/ceg5101/oflops/cbench'
Making install in doc
make[1]: Entering directory '/home/ceg5101/oflops/doc'
make[1]: Nothing to be done for 'install'.
make[1]: Leaving directory '/home/ceg5101/oflops/doc'
Enjoy Mininet!
```

16. To test the mininet, type "`sudo mn`" and you will see a small network of 2 hosts connected with a single switch and a controller is created, and the mininet is initiated as shown below.



17. This confirms the successful installation of the mininet.
18. Enter "`exit`" to quit
19. It is optional but recommended to sign up for the mininet-discuss mailing list at https://mailman.stanford.edu/mailman/listinfo/mininet-discuss to get support and discuss with the mininet community.

Task 2: To become familiar with the Mininet Interface

1. In this activity, we will become familiar with the Mininet environment and start with building a simple network by connecting two end devices with the switch.

2. After completing the installation, launch a terminal and type "`sudo mn -h`" to get help describing Mininet options and commands. This gives a list of commands to be used to create a network emulation.

https://github.com/mininet/mininet/wiki/Introduction-to-Mininet
http://mininet.org/walkthrough/

```
ceg5101@ceg5101-VirtualBox:~$ sudo mn -h
Usage: mn [options]
(type mn -h for details)

The mn utility creates Mininet network from the command line. It can create
parametrized topologies, invoke the Mininet CLI, and run tests.

Options:
  -h, --help            show this help message and exit
  --switch=SWITCH       default|ivs|lxbr|ovs|ovsbr|ovsk|user[,param=value...]
                        user=UserSwitch ovs=OVSSwitch ovsbr=OVSBridge
                        ovsk=OVSSwitch ivs=IVSSwitch lxbr=LinuxBridge
                        default=OVSSwitch
  --host=HOST           cfs|proc|rt[,param=value...] proc=Host
                        rt=CPULimitedHost{'sched': 'rt'}
                        cfs=CPULimitedHost{'sched': 'cfs'}
  --controller=CONTROLLER
                        default|none|nox|ovsc|ref|remote|ryu[,param=value...]
                        ref=Controller ovsc=OVSController nox=NOX
                        remote=RemoteController ryu=Ryu
                        default=DefaultController none=NullController
  --link=LINK           default|ovs|tc|tcu[,param=value...] default=Link
                        tc=TCLink tcu=TCULink ovs=OVSLink
  --topo=TOPO           linear|minimal|reversed|single|torus|tree[,param=value
                        ...] minimal=MinimalTopo linear=LinearTopo
                        reversed=SingleSwitchReversedTopo
                        single=SingleSwitchTopo tree=TreeTopo torus=TorusTopo
  -c, --clean           clean and exit
  --custom=CUSTOM       read custom classes or params from .py file(s)
  --test=TEST           pingall|pingpair|iperf|iperfudp|all|none|build
  -x, --xterms          spawn xterms for each node
```

3. Following the above help, the command to build a custom topology of a single switch and two hosts is "`sudo mn --topo single,2`"

```
ceg5101@ceg5101-VirtualBox:~$ sudo mn --topo single,2
*** Creating network
*** Adding controller
*** Adding hosts:
h1 h2
*** Adding switches:
s1
*** Adding links:
(h1, s1) (h2, s1)
*** Configuring hosts
h1 h2
*** Starting controller
c0
*** Starting 1 switches
s1 ...
*** Starting CLI:
mininet>
```

4. The default topology is a minimal topology, and it can be changed with the command "`--topo=NAME`". The available NAME for defining different topologies can be found from the output of "`sudo mn -h`" in step 2.2.

5. To find out more about the current network topology, type "`net`" and enter. The window is shown below. It shows that port **eth0** of host **h1** is connected to port **eth1** of switch **s1.** Similarly, port **eth0** of host **h2** is connected to **eth2** of a switch **s1**. Also, switch **s1** has a loopback interface **lo**.

```
mininet> net
h1 h1-eth0:s1-eth1
h2 h2-eth0:s1-eth2
s1 lo:  s1-eth1:h1-eth0 s1-eth2:h2-eth0
c0
mininet>
```

https://github.com/mininet/mininet/wiki/Introduction-to-Mininet
http://mininet.org/walkthrough/

6. Mininet allows users to issue a specific command on a device with the format of "DEVICE-NAME COMMAND". The first string typed into Mininet CLI is the name of the host, controller, or switch, and the command typed next to the device name is the one executed on that node. As an example, to check the configured IP address for the host h1, type "`h1 ifconfig`", and for host h2 type "`h2 ifconfig`". To print the process list from host h1 type "`h1 ps -a`" and so on.

```
mininet> h1 ifconfig
h1-eth0: flags=4163<UP,BROADCAST,RUNNING,MULTICAST>  mtu 1500
        inet 10.0.0.1  netmask 255.0.0.0  broadcast 10.255.255.255
        inet6 fe80::20f3:9eff:fe9b:ea24  prefixlen 64  scopeid 0x20<link>
        ether 22:f3:9e:9b:ea:24  txqueuelen 1000  (Ethernet)
        RX packets 31  bytes 3512 (3.5 KB)
        RX errors 0  dropped 0  overruns 0  frame 0
        TX packets 9  bytes 726 (726.0 B)
        TX errors 0  dropped 0 overruns 0  carrier 0  collisions 0

lo: flags=73<UP,LOOPBACK,RUNNING>  mtu 65536
        inet 127.0.0.1  netmask 255.0.0.0
        inet6 ::1  prefixlen 128  scopeid 0x10<host>
        loop  txqueuelen 1000  (Local Loopback)
        RX packets 0  bytes 0 (0.0 B)
        RX errors 0  dropped 0  overruns 0  frame 0
        TX packets 0  bytes 0 (0.0 B)
        TX errors 0  dropped 0 overruns 0  carrier 0  collisions 0

mininet>
```

7. To find and explore more on available Mininet commands, type "`help`". As an example, the command "`nodes`" display nodes, "links" displays links and their active status, and so on.

```
mininet> help

Documented commands (type help <topic>):
========================================
EOF    gterm  iperfudp  nodes        pingpair      py      switch  xterm
dpctl  help   link      noecho       pingpairfull  quit    time
dump   intfs  links     pingall      ports         sh      wait
exit   iperf  net       pingallfull  px            source  x

You may also send a command to a node using:
  <node> command {args}
For example:
  mininet> h1 ifconfig

The interpreter automatically substitutes IP addresses
for node names when a node is the first arg, so commands
like
  mininet> h2 ping h3
should work.

Some character-oriented interactive commands require
noecho:
  mininet> noecho h2 vi foo.py
However, starting up an xterm/gterm is generally better:
  mininet> xterm h2

mininet>
```

https://github.com/mininet/mininet/wiki/Introduction-to-Mininet
http://mininet.org/walkthrough/

8. You can check network connectivity by sending a ping command from one host to another (`control + C` to quit the ping process).

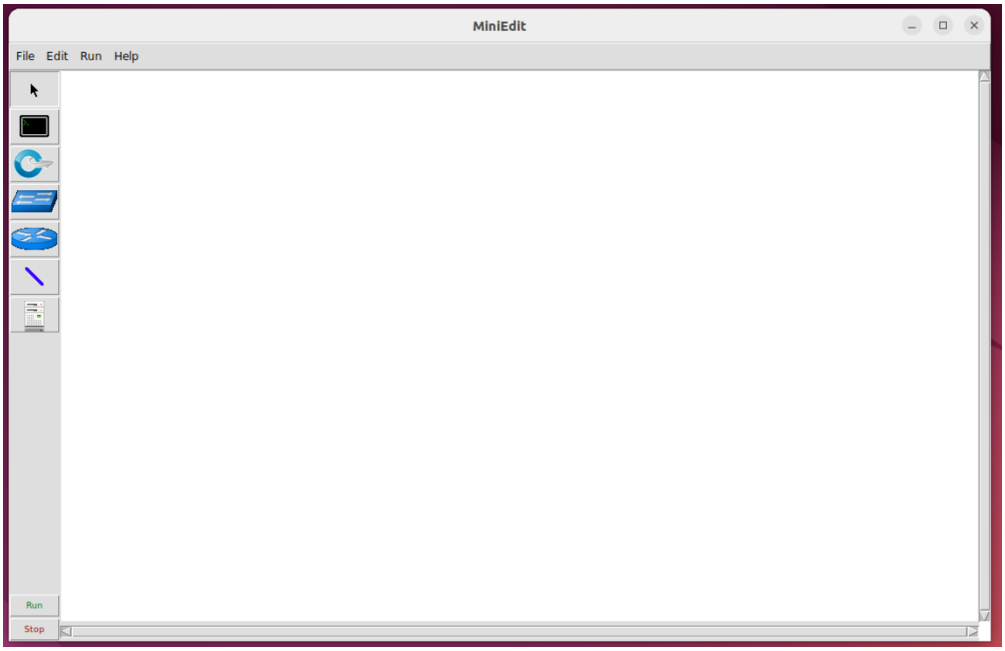
```
mininet> h1 ping h2
PING 10.0.0.2 (10.0.0.2) 56(84) bytes of data.
64 bytes from 10.0.0.2: icmp_seq=1 ttl=64 time=30.9 ms
64 bytes from 10.0.0.2: icmp_seq=2 ttl=64 time=0.311 ms
64 bytes from 10.0.0.2: icmp_seq=3 ttl=64 time=0.058 ms
64 bytes from 10.0.0.2: icmp_seq=4 ttl=64 time=0.068 ms
64 bytes from 10.0.0.2: icmp_seq=5 ttl=64 time=0.048 ms
```

9. To leave Mininet and end the current network setup, type "`exit`".

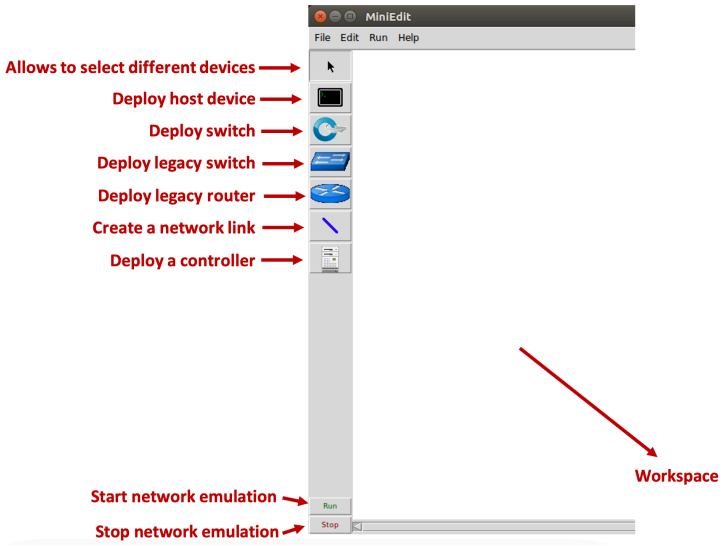
```
56 packets transmitted, 56 received, 0% packet loss, time
rtt min/avg/max/mdev = 0.047/0.639/30.946/4.087 ms
mininet> exit
*** Stopping 1 controllers
c0
*** Stopping 2 links
..
*** Stopping 1 switches
s1
*** Stopping 2 hosts
h1 h2
*** Done
completed in 224.641 seconds
ceg5101@ceg5101-VirtualBox:~$
```

10. Next, we will learn to use the GUI interface of Mininet, known as MiniEdit.
11. To get started with MiniEdit, locate the folder where mininet is installed and type "`sudo python3 mininet/examples/miniedit.py`". The default interface window for MiniEdit is shown below. (if you are using python2, you can use the command: "`sudo ~/mininet/examples/miniedit.py`")

https://github.com/mininet/mininet/wiki/Introduction-to-Mininet
http://mininet.org/walkthrough/



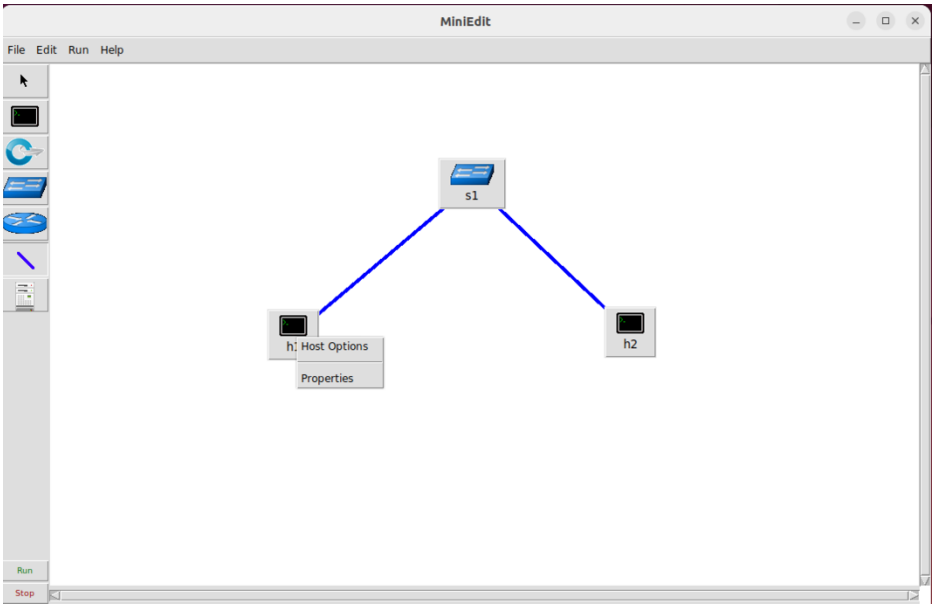
1. You can move your cursor around different icons and a help option will appear with an explanatory title. Some of the key interface bars and their functioning is highlighted below.



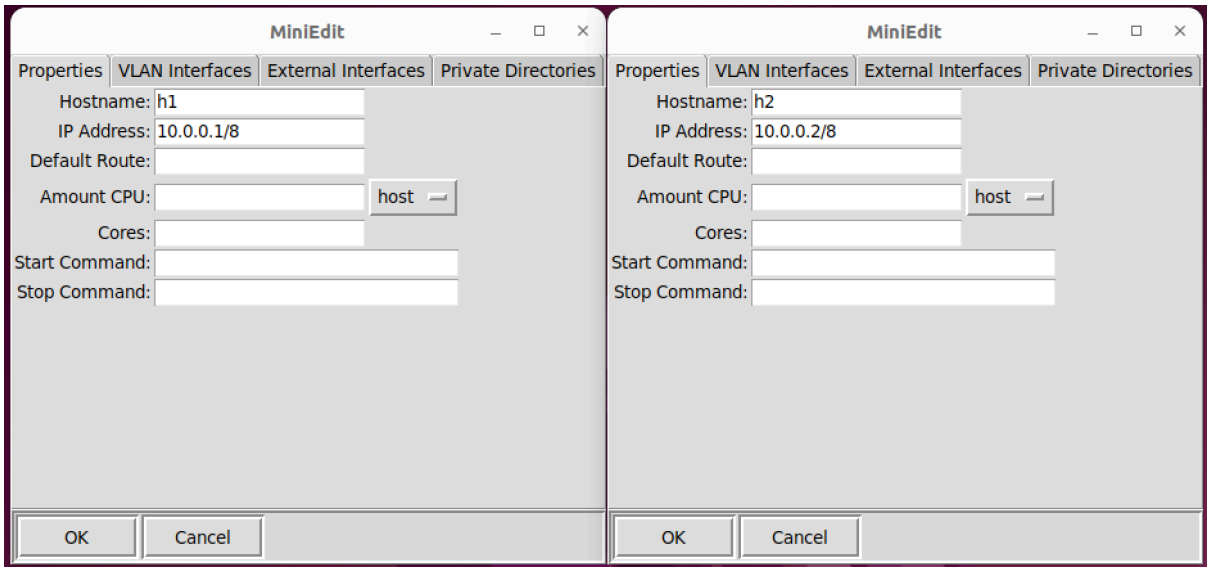
2. We can add devices to the workspace by selecting the device type (here - LEGACY SWITCH s1, HOST DEVICE h1 and h2) from the sidebar and clicking it on the workspace. A link between two devices can be created by dragging the link option from one device and dropping it on another device.

3. Next, to configure a device, right-click on the icon, and select the properties.

https://github.com/mininet/mininet/wiki/Introduction-to-Mininet
http://mininet.org/walkthrough/



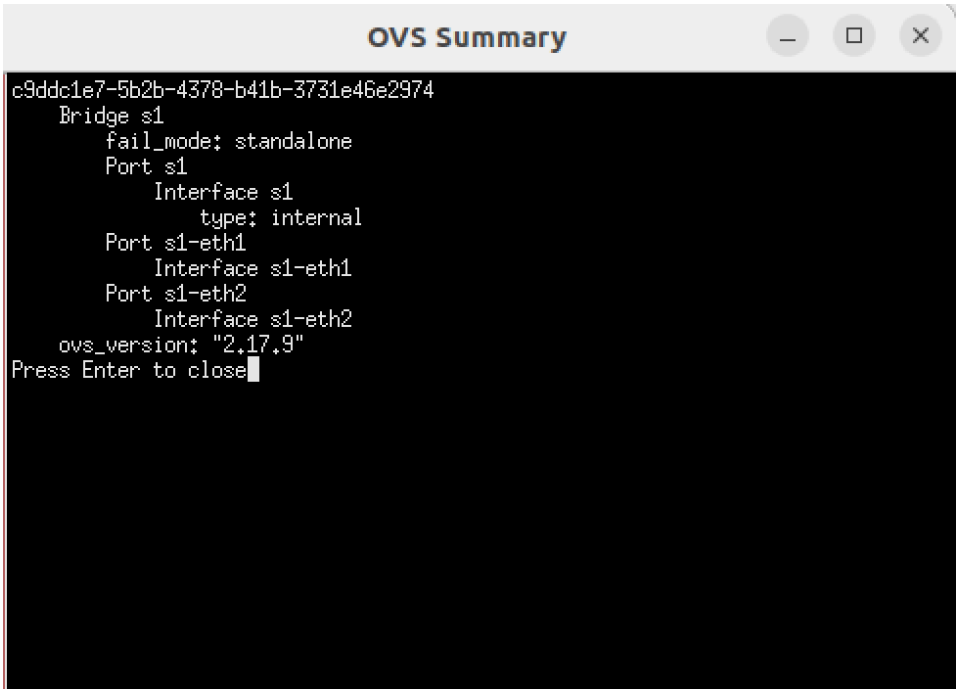
4. Below shows the property window of host h1 and switch s1, to add or remove different parameters to the devices. We can configure the IP address of host h1 and host h2 using the device property window. As an example here, we use 10.0.0.1/8 for host h1 and 10.0.0.2/8 for host h2.



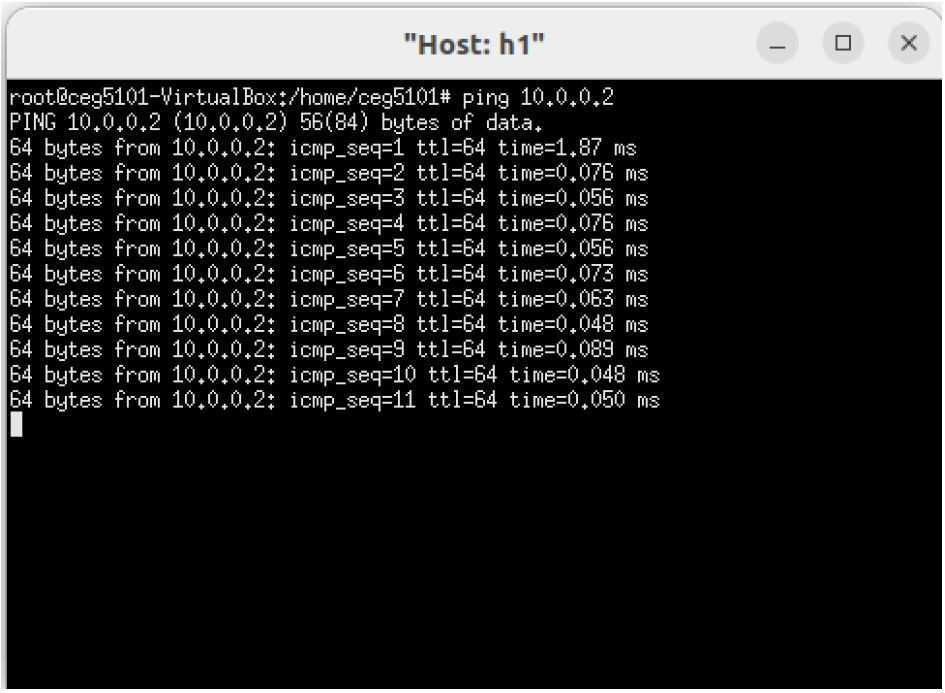
5. To start the simulation, click on the "Run" button.

6. After starting the simulation, we can view the network summary by clicking "Show OVS summary" from the "Run" tab.

https://github.com/mininet/mininet/wiki/Introduction-to-Mininet
http://mininet.org/walkthrough/



7. We can also check connectivity between devices by sending a ping command from one host to another. To do so, right-click on the host and select terminal. Type ping from one host with the address of another host.



8. Finally, to save your file as a completed network and for future use, click on File>save and then save it with .mn (mininet activity file) file format.

https://github.com/mininet/mininet/wiki/Introduction-to-Mininet
http://mininet.org/walkthrough/

Saved File Reload Issue

A. Some of you may face the problem of failing to load saved mn file

Here are the solutions to solve it.

The problem is rooted on line 1423 in /mininet/examples/miniedit.py:

```
unicode = globals.get( 'unicode', str)
```

Modified code for your review.

```
$ cd mininet/examples/
$ nano miniedit.py
```

Replace with following code

```
def convertJsonUnicode(self, text):
        "Some part of Mininet don't like Unicode"
        unicode = globals().get('str', bytes)
        if isinstance(text, dict):
              return {self.convertJsonUnicode(key):
              self.convertJsonUnicode(value) for key, value in text.items()}
        if isinstance(text, list):
            return [self.convertJsonUnicode(element) for element in text]
        if isinstance(text, unicode):  # pylint: disable=undefined-variable
            return text.encode('utf-8')
        return text
```

We believe this adjustment will ensure the loading of saved files.

B. Heck for the following error in the terminal

```
topo=none
Exception in Tkinter callback
Traceback (most recent call last):
  File "/usr/lib/python3.8/tkinter/__init__.py", line 1892, in __call__
    return self.func(*args)
  File "mininet/examples/miniedit.py", line 1486, in loadTopology
    hosts = loadedTopology['hosts']
KeyError: 'hosts'
```

Press Ctrl + W and search “convertjson”. It should go to the function **convertJsonUnicode**. Go down to the function **loadTopology**.

```
def loadTopology( self ):
    "Load command."
    c = self.canvas

    myFormats = [
        ('Mininet Topology','*.mn'),
        ('All Files','*'),
    ]
    f = tkFileDialog.askopenfile(filetypes=myFormats, mode='rb')
    if f is None:
        return
    self.newTopology()
    loadedTopology = self.convertJsonUnicode(json.load(f))

    # Load application preferences
    if 'application' in loadedTopology:
        self.appPrefs.update(loadedTopology['application'])
        if "ovsOf10" not in self.appPrefs["openFlowVersions"]:
            self.appPrefs["openFlowVersions"]["ovsOf10"] = '0'
```

Change the highlighted line by removing the function call ‘self.convertJsonUnicode’.

```
def loadTopology( self ):
    "Load command."
    c = self.canvas

    myFormats = [
        ('Mininet Topology','*.mn'),
        ('All Files','*'),
    ]
    f = tkFileDialog.askopenfile(filetypes=myFormats, mode='rb')
    if f is None:
        return
    self.newTopology()
    loadedTopology = json.load(f)

    # Load application preferences
    if 'application' in loadedTopology:
        self.appPrefs.update(loadedTopology['application'])
        if "ovsOf10" not in self.appPrefs["openFlowVersions"]:
            self.appPrefs["openFlowVersions"]["ovsOf10"] = '0'
```